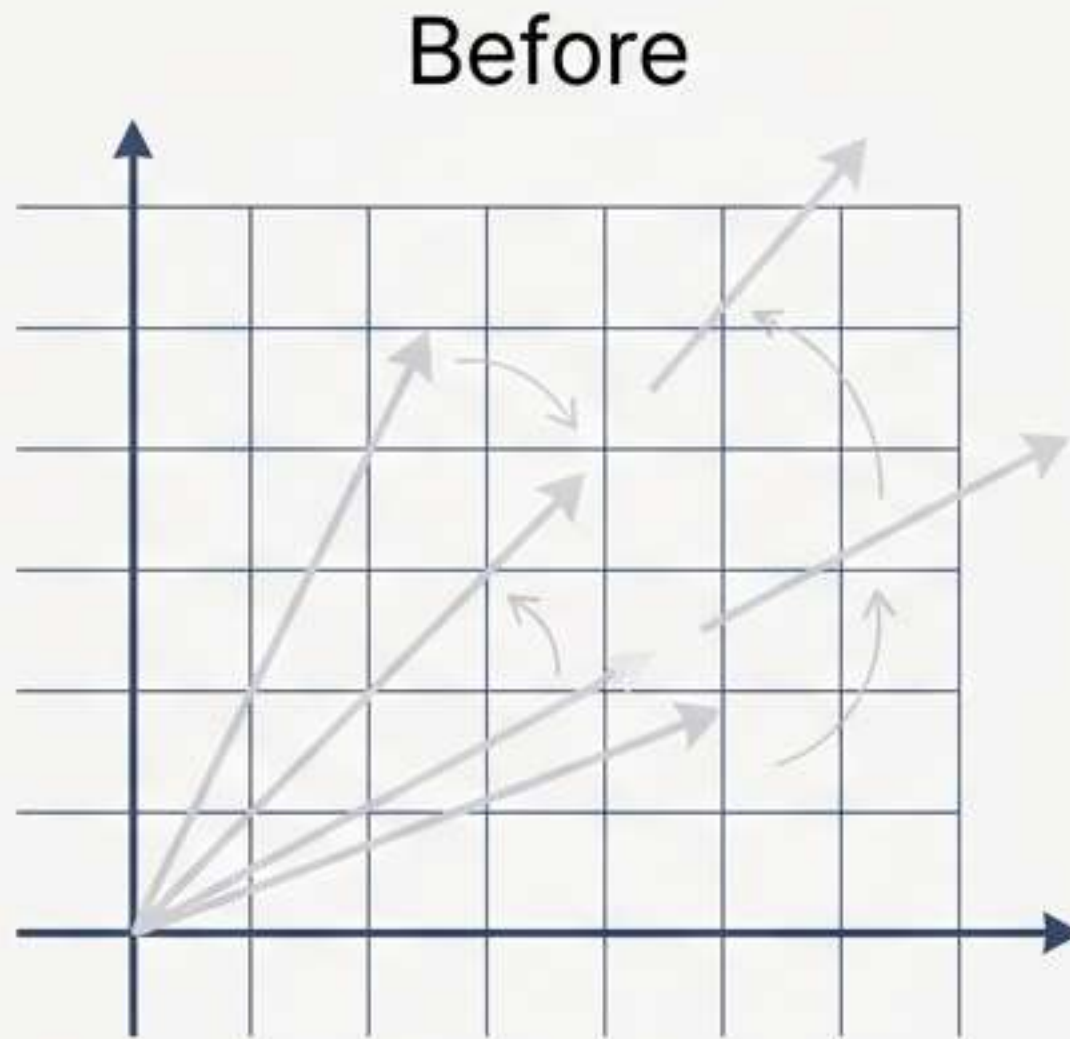


Mastering Eigenvalues & Eigenvectors

From Linear Algebra to Machine Learning

**Most vectors change direction during a transformation.
A special few do not.**



Geometric Intuition

When a matrix transforms a space, it creates geometric chaos. However, certain vectors remain fixed on their original span. They are only stretched or compressed.

Defining the invariant direction and its scaling factor.

$$Av = \lambda v$$

Eigenvector (v)

A non-zero vector whose direction remains perfectly unchanged when a linear transformation is applied.

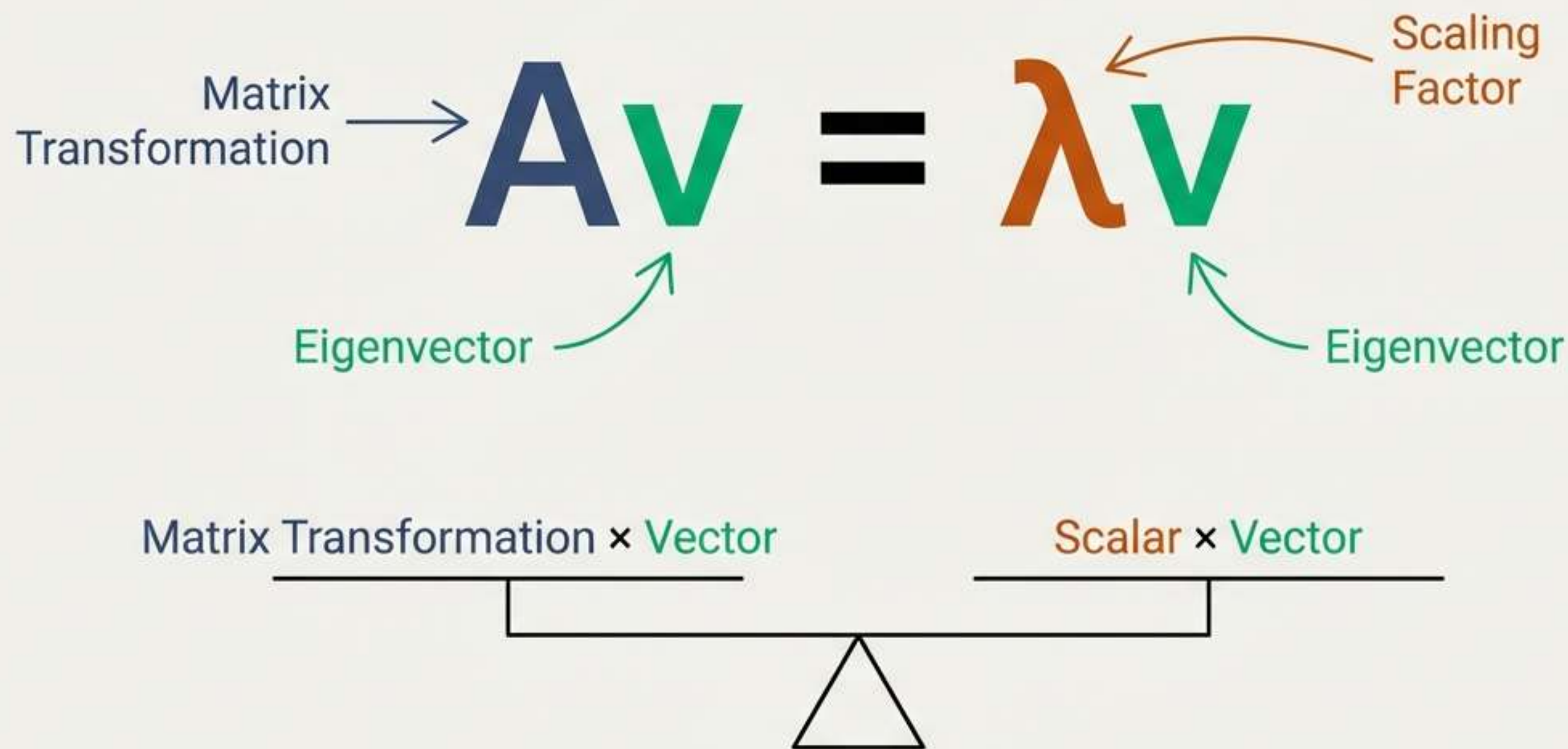
Eigenvalue (λ)

The scalar value that dictates exactly how much the Eigenvector is stretched, compressed, or flipped.

Data Science Rule

Multiplying a matrix by this vector does not change its direction, it only scales it by factor λ .

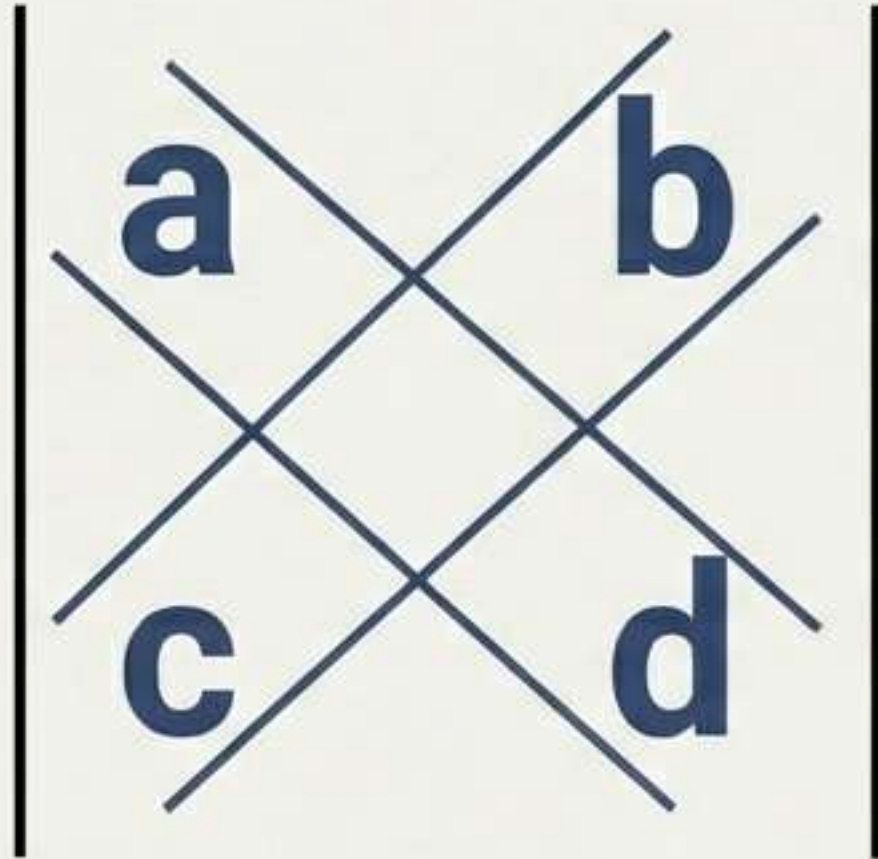
The foundational equation linking matrices to scalars



The magic of this equation lies in its simplicity. A complex matrix operation (left) is entirely equivalent to a simple scalar multiplication (right), provided we are moving along the Eigenvector.

The mathematical prerequisite: calculating determinants.

Before finding Eigenvalues, we must master the determinant—a value denoting the scaling factor of the linear transformation.



For a 2×2 matrix:

$$|A| = (a \times d) - (b \times c)$$

For larger 3x3 matrices, we apply the strict sign rule to the top row (+, -, +) before multiplying the subsequent sub-matrices.

Deriving the Characteristic Equation

If the Eigenvector ($\mathbf{v}$) is zero, the equation is meaningless. To guarantee a non-trivial solution where $\mathbf{v} \neq \mathbf{0}$, the **determinant** of our new matrix ($\mathbf{A} - \lambda \mathbf{I}$) must be exactly zero.

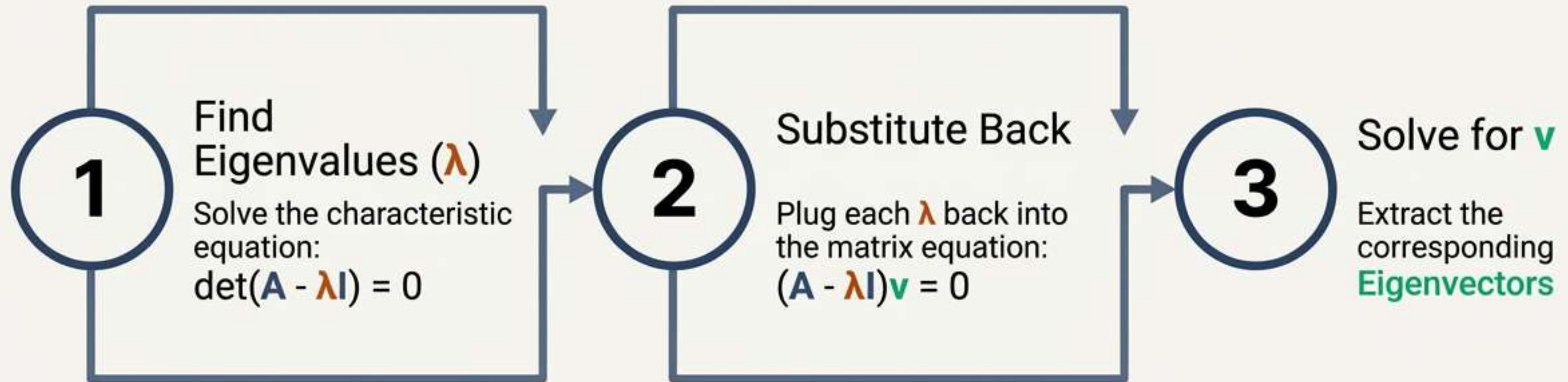
1. Start with: $(\mathbf{A} - \lambda \mathbf{I})\mathbf{v} = \mathbf{0}$

2. For non-trivial solution ($\mathbf{v} \neq \mathbf{0}$):


$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0$$

This is called the **Characteristic Equation**. Its roots are the **Eigenvalues** (λ).

The three-step algorithmic blueprint.



Step 1: Solving the characteristic equation.

Given Matrix A: $\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$

$$\text{Equation: } \det\left(\begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix}\right) = 0$$

$$(4-\lambda)(3-\lambda) - 2 = 0$$



$$\lambda^2 - 7\lambda + 10 = 0$$

The roots (Eigenvalues) are $\lambda_1 = 5$ and $\lambda_2 = 2$.

Step 2: Extracting the first Eigenvector.

Action: Substitute $\lambda_1 = 5$ into $(A - 5I)\mathbf{v} = \mathbf{0}$.

Maths:
$$\begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0}$$

Equation reduction (Simultaneous equations):

$$-x + y = 0 \Rightarrow y = x$$

The first Eigenvector $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Step 3: Extracting the second Eigenvector.

Action: Substitute $\lambda_2 = 2$ into $(A - 2I)\mathbf{v} = 0$.

Maths:
$$\begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0}$$

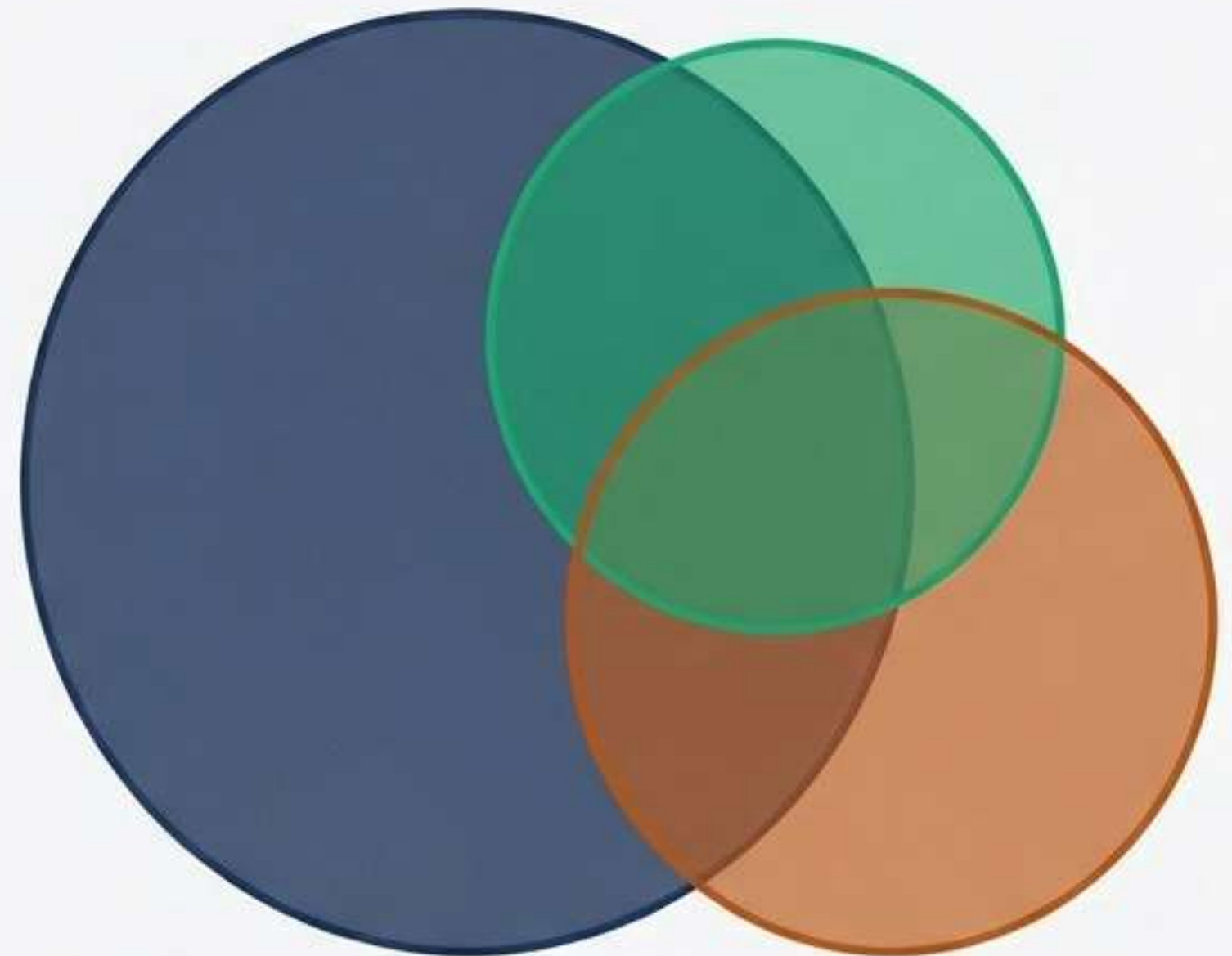
Equation reduction: $2x + y = 0 \Rightarrow y = -2x$

The second Eigenvector $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$.

Translating pure mathematics into Data Science.

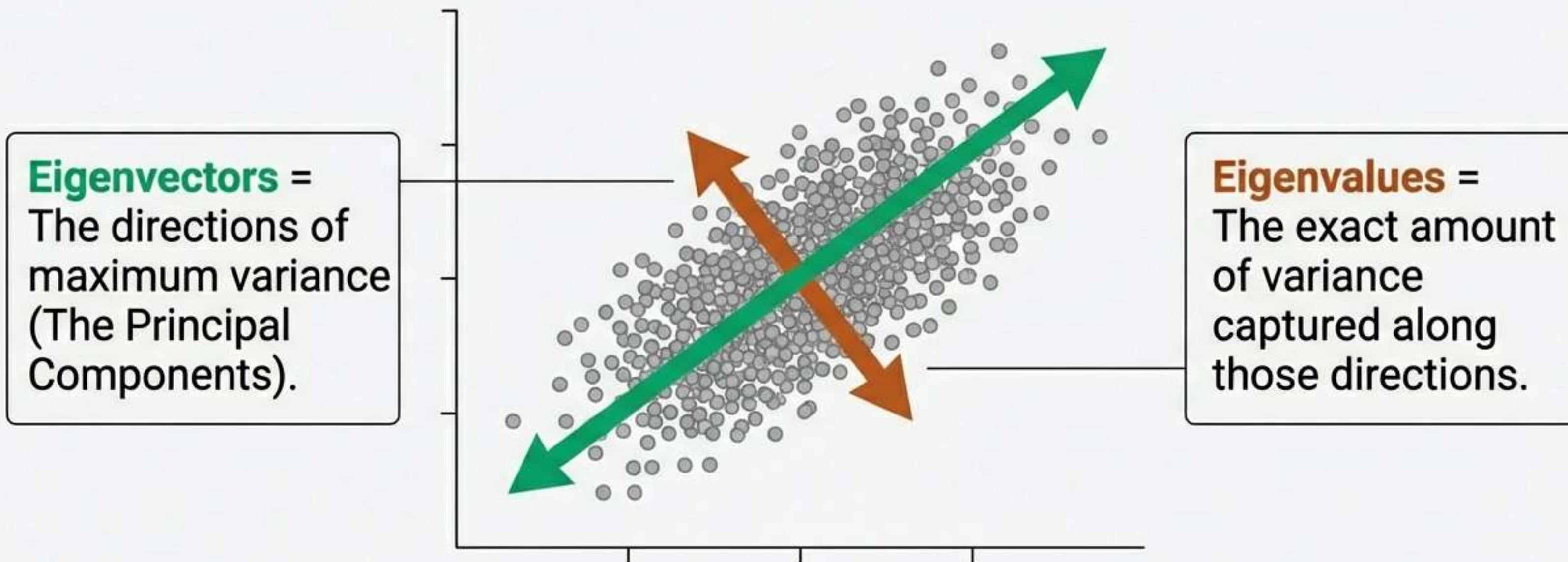
The maths is proven. But for Data Science practitioners, three vital questions remain:
Roboto

- **Why do we learn them?**
(To understand the underlying structure of high-dimensional data).
- **When do we apply them?**
(When a dataset contains too many correlated variables).
- **Where do we apply them?**
(Dimensionality reduction algorithms).

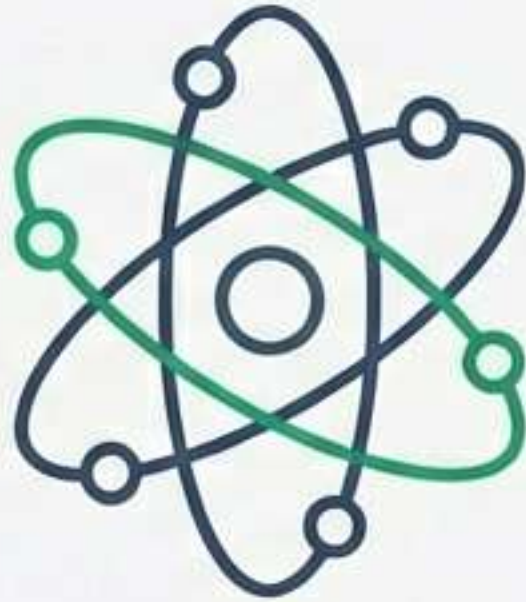


Dimensionality Reduction & PCA

In Machine Learning, we convert raw data matrices into Eigenvectors to simplify complex datasets while retaining their core information.



Beyond PCA: Real-world applications across sciences.



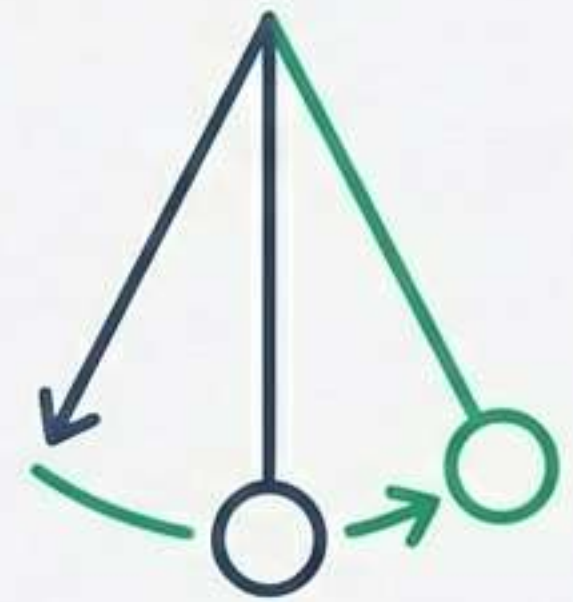
Quantum Mechanics

Used to calculate energy states and observe quantum system behaviours.



Graph Analysis

Powers algorithms that determine network connectivity, stability, and node importance (like PageRank).



Stability Analysis

Used in mechanical engineering and control theory to determine if a system will safely return to equilibrium.

The Ultimate Eigen-Cheat Sheet.

The Formula

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0$$

The Intuition

- **Eigenvalues** are scaling factors.
- **Eigenvectors** are invariant directions.

The Algorithm

1. Find Characteristic Equation → 2. Solve for λ → 3. Solve for $\mathbf{v}$

ML Utility

Drives Principal Component Analysis (**PCA**) for dimensionality reduction.

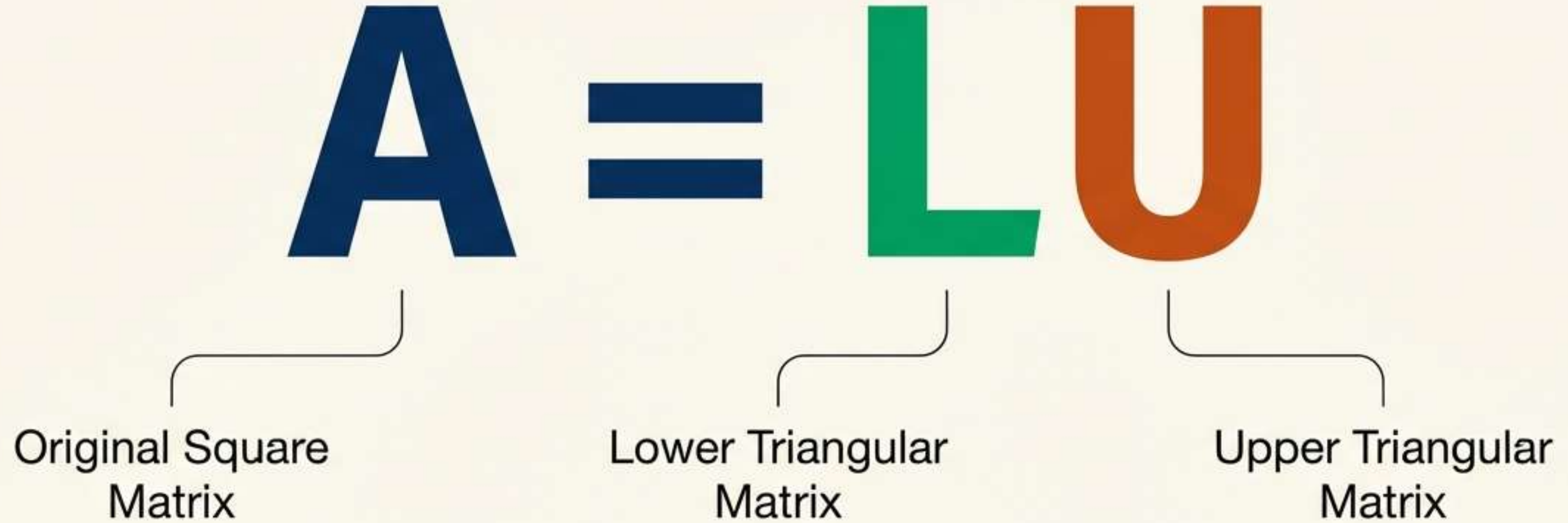
Mastery Unlocked.

You have successfully bridged the gap
between pure linear algebra and
applied machine learning.

LU Decomposition

Numerical Linear Algebra for Data Science

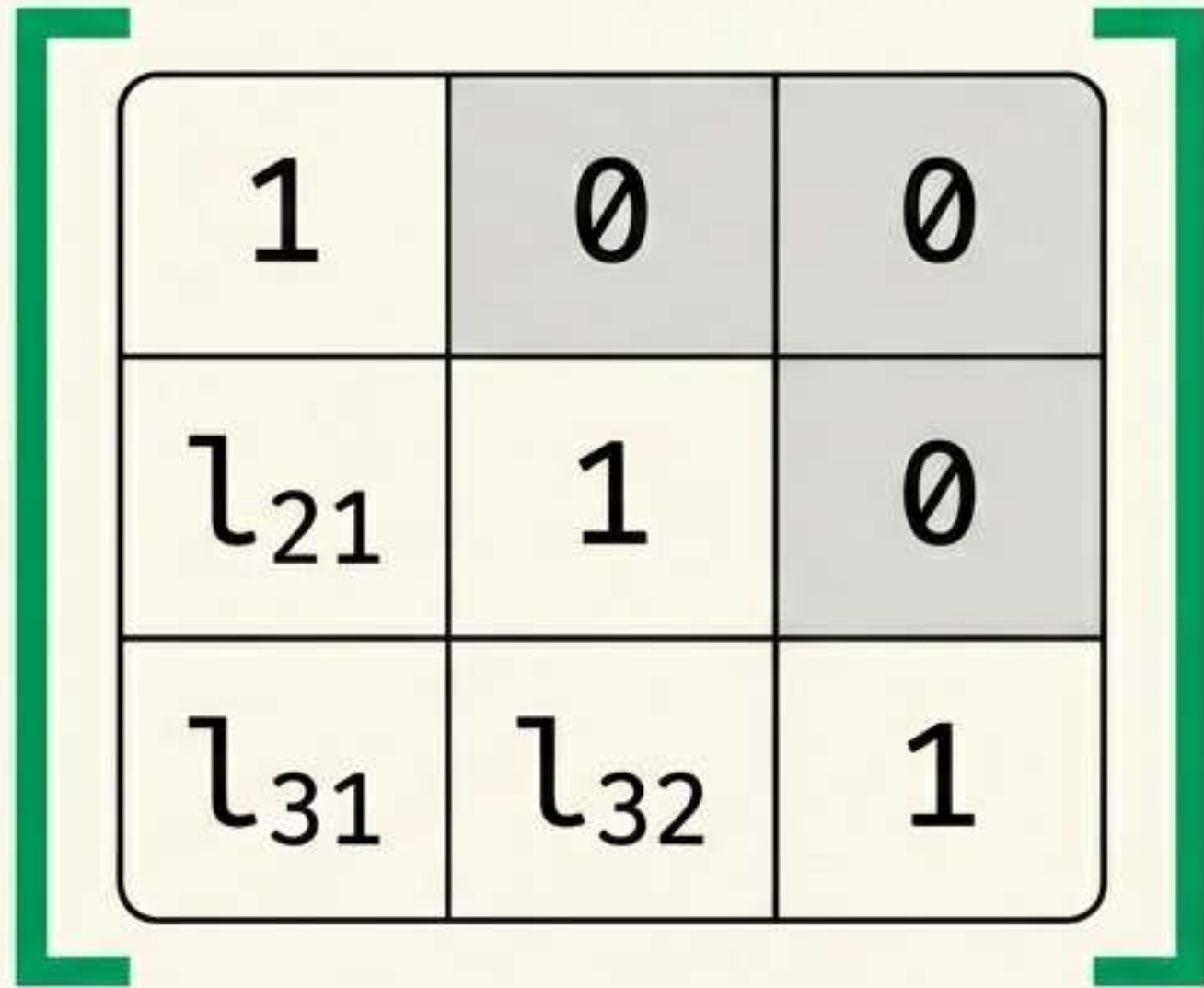




- LU decomposition is a matrix factorisation technique.

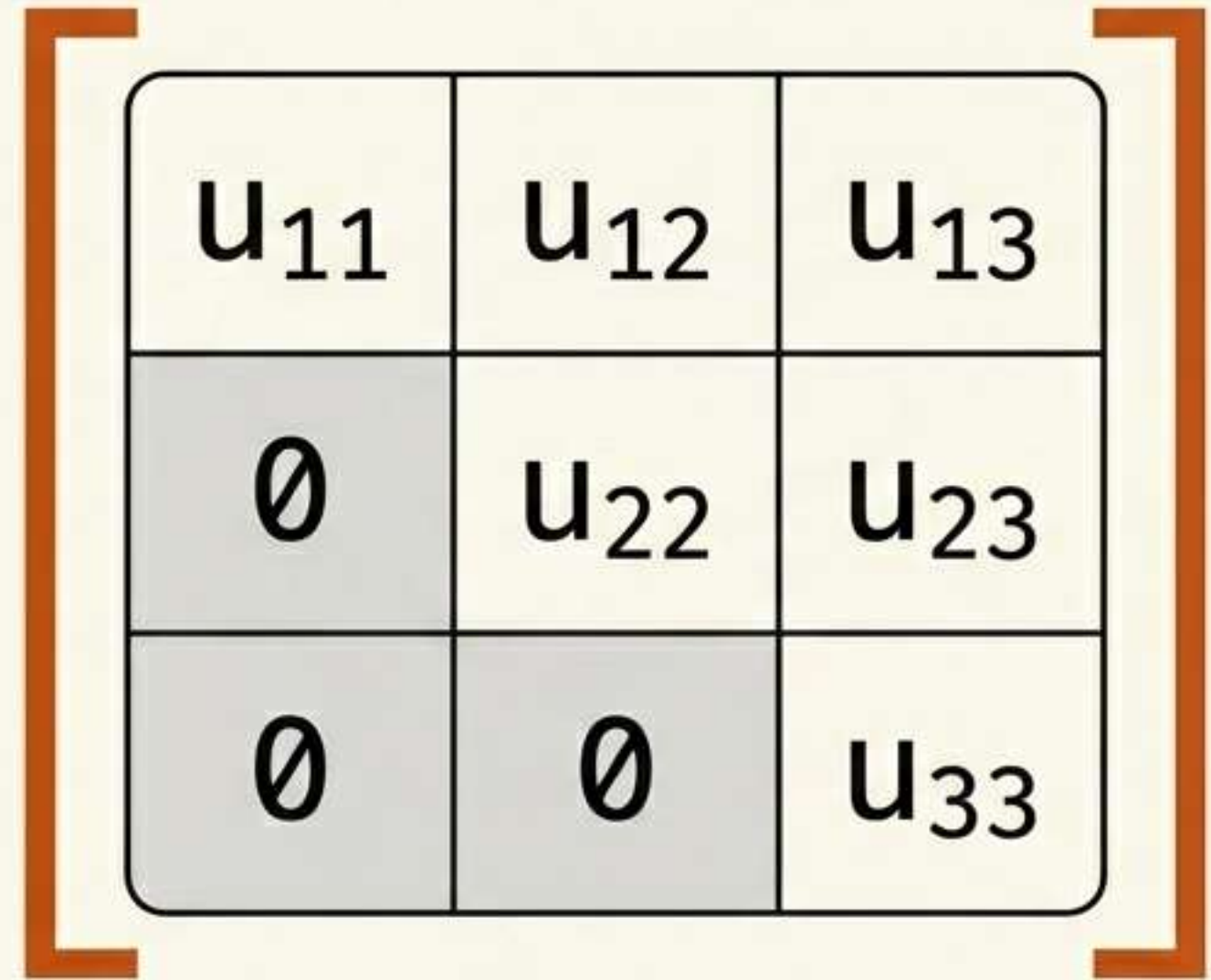
- It breaks down a complex square matrix into the product of two simpler, purpose-built matrices.

The Anatomy of Triangular Matrices

A 3x3 matrix enclosed in green brackets. The diagonal elements are 1, 1, and 1. The elements below the diagonal are l₂₁, l₃₁, and l₃₂. The elements above the diagonal are 0, 0, and 0.

1	0	0
l_{21}	1	0
l_{31}	l_{32}	1

Lower Triangular (L): All entries above the diagonal are zero.
(Diagonal elements are typically set to 1 to simplify computation).

A 3x3 matrix enclosed in orange brackets. The diagonal elements are u₁₁, u₂₂, and u₃₃. The elements below the diagonal are 0, 0, and 0. The elements above the diagonal are u₁₂, u₁₃, and u₂₃.

u_{11}	u_{12}	u_{13}
0	u_{22}	u_{23}
0	0	u_{33}

Upper Triangular (U): All elements below the diagonal are zero.

The Computational Advantage of Triangular Matrices

$$\begin{bmatrix} 2 & 7 & 9 \\ 0 & 4 & 1 \\ 0 & 0 & 3 \end{bmatrix} \rightarrow 2 \times 4 \times 3 = 24$$

Simple Determinants:

For any triangular matrix, the determinant is simply the product of its diagonal entries. No complex co-factor expressions required.

$$\begin{bmatrix} 5 & 0 & 0 \\ 8 & 3 & 0 \\ 4 & 6 & 2 \end{bmatrix} \rightarrow 5 \times 3 \times 2 = 30$$

Preserved Structure:

Multiplying two upper (or lower) triangular matrices yields another matrix of the same type.

The Core Goal: Solving Systems of Linear Equations

The diagram shows the equation $Ax = b$ with three terms. Below each term is a label connected by a bracket. The label for A is "Coefficient Matrix (a square matrix)". The label for x is "Column Vector of Unknowns (x1, x2, x3)". The label for b is "Column Vector of Constants".

$$Ax = b$$

Coefficient Matrix
(a square matrix)

Column Vector of
Unknowns (x1, x2, x3)

Column Vector
of Constants

Solving $Ax = b$ directly for large systems is computationally expensive.
By decomposing A into L and U , we can bypass direct, inefficient calculation.

The Two-Step LU Workflow

Step 0:
 $\mathbf{Ax} = \mathbf{b}$



Substitute
 $\mathbf{LU}$ for $\mathbf{A}$



$$(\mathbf{LU})\mathbf{x} = \mathbf{b}$$

Step 1:

Let $\mathbf{Ux} = \mathbf{y}$.

Solve $\mathbf{Ly} = \mathbf{b}$ to find vector $\mathbf{y}$.

Step 2:

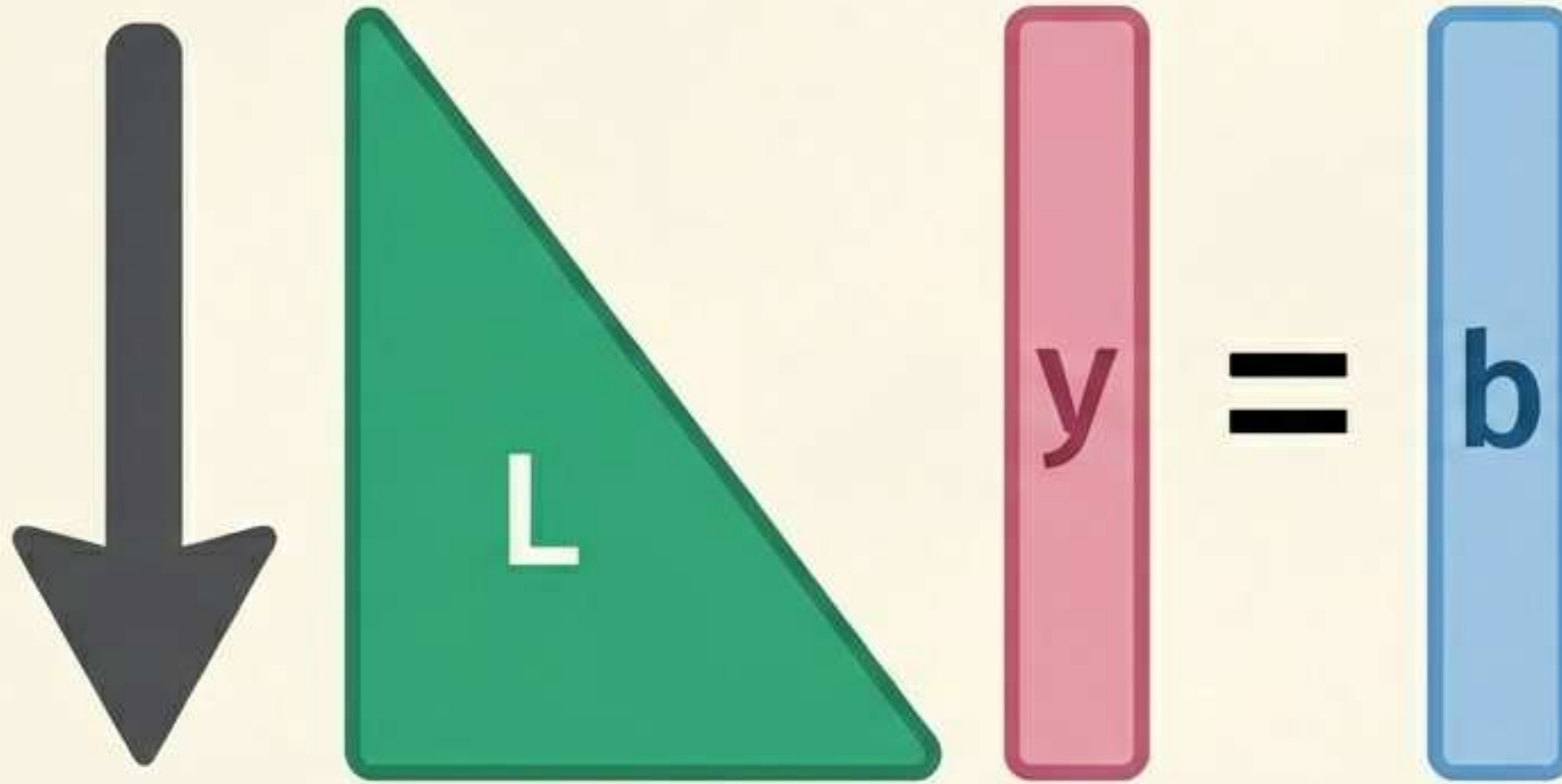
Solve $\mathbf{Ux} = \mathbf{y}$ to find the final vector $\mathbf{x}$.

The Recipe

1. $\mathbf{Ly} = \mathbf{b}$

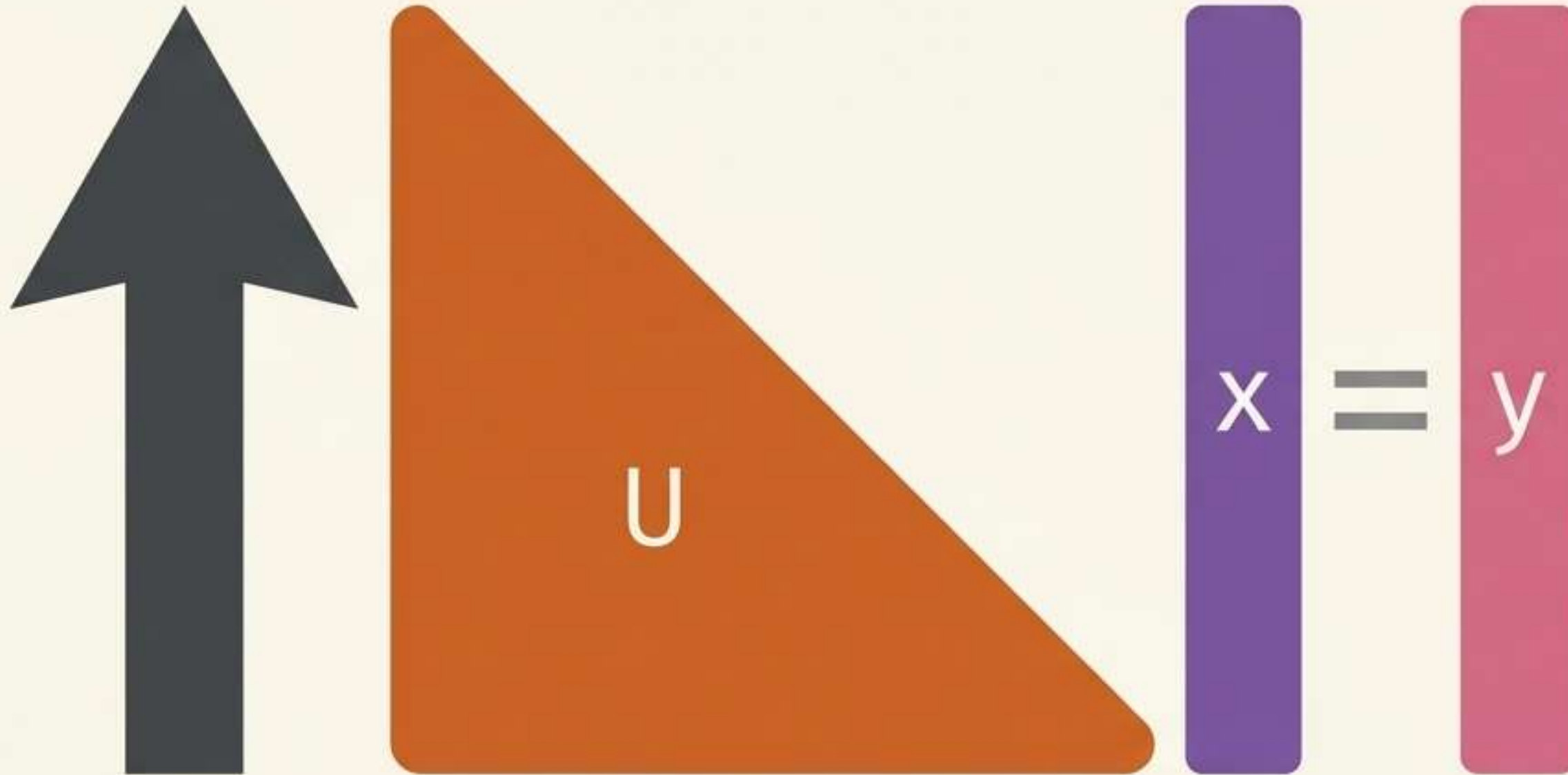
2. $\mathbf{Ux} = \mathbf{y}$

Step 1: Forward Substitution ($Ly = b$)



- Used for Lower Triangular systems.
- Top-Down Process: Start by solving for the first variable (y_1) from the top equation.
- Substitute the computed value downwards into successive equations.
- Each row only involves variables already calculated.

Step 2: Backward Substitution ($Ux = y$)



- Used for **Upper Triangular** systems.
- **Bottom-Up Process:** Start by solving for the last variable (x_n) from the bottom equation.
- Substitute the computed value upwards into preceding equations.
- Each row only involves variables not yet computed.

Case Study: Defining the 3x3 System

$$\begin{cases} 2X_1 + 4X_2 - 2X_3 = 6 \\ 4X_1 + 9X_2 - 3X_3 = 8 \\ -2X_1 - 3X_2 + 7X_3 = 10 \end{cases}$$

Standard Algebraic Form

You Are
Here →

The Recipe

1. Setup A

2. Solve y

3. Solve x

Matrix Form ($Ax = b$)

$$\begin{bmatrix} \\ \\ \end{bmatrix} \begin{bmatrix} \\ \\ \end{bmatrix} = \begin{bmatrix} \\ \\ \end{bmatrix}$$

Factorising **A** into **L** and **U**

$$\begin{matrix} & \mathbf{A} \\ \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} & = & \begin{matrix} & \mathbf{L} \\ \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \end{matrix} \times & \begin{matrix} & \mathbf{U} \\ \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} \end{matrix} \end{matrix}$$

- We compute elements directly by multiplying Matrices **L** and **U** and matching the results to the known coefficients in Matrix **A**.
- By isolating rows sequentially, we substitute values until all elements in **L** and **U** are found.

The Recipe

1. Setup **A**

2. Solve **y**

3. Solve **x**

Executing Forward Substitution ($\mathbf{Ly} = \mathbf{b}$)

Row 1 yields: $y_1 = 5$



Row 2 yields: $y_2 = -4$



Row 3 yields: $y_3 = 4$

Final Result: Vector $\mathbf{y} = \begin{bmatrix} 5 \\ -4 \\ 4 \end{bmatrix}^T$

The Recipe

1. Setup $\mathbf{A}$

2. Solve $\mathbf{Ly} = \mathbf{b}$

3. Solve $\mathbf{Ux} = \mathbf{y}$

Executing Backward Substitution ($Ux = y$)

Row 1 yields: $x_1 = 1.7$



Row 2 yields: $x_2 = 0.8$



Row 3 yields: $x_3 = -0.8$

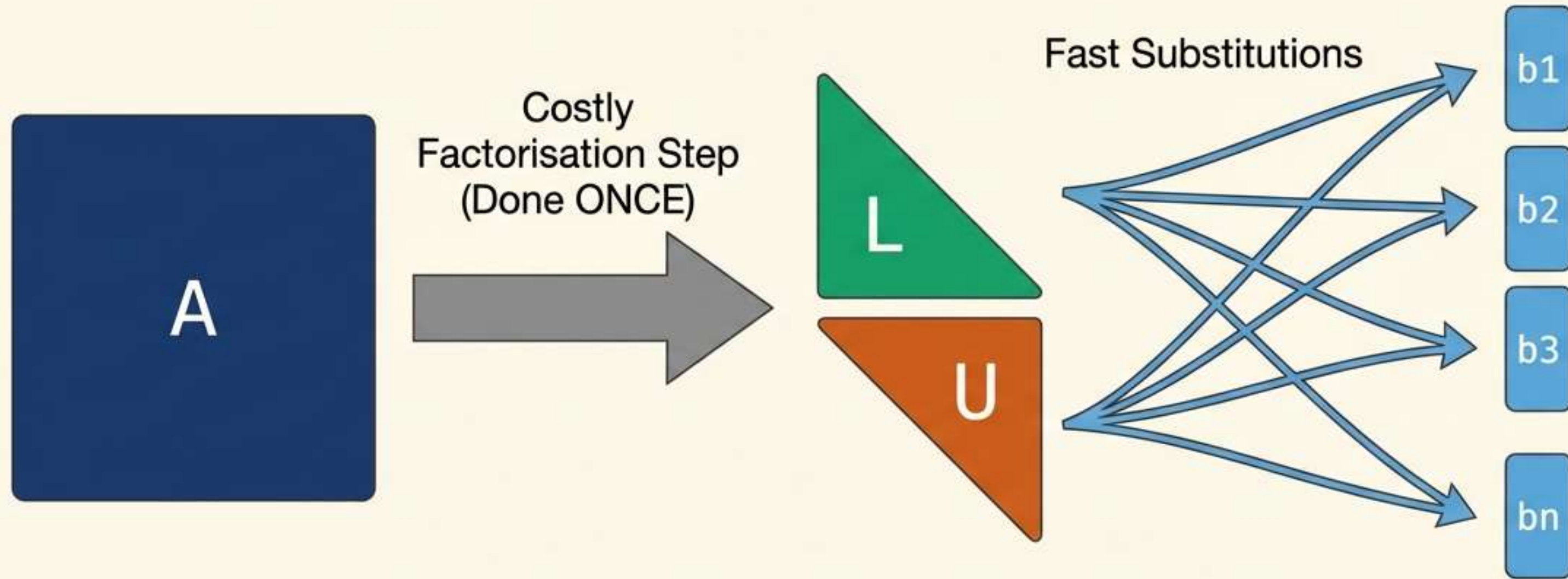
Final Result: Vector

$$\mathbf{x} = [1.7, 0.8, -0.8]^T$$

The Recipe

1. Setup A
2. Solve $Ly = b$
3. Solve $Ux = y$

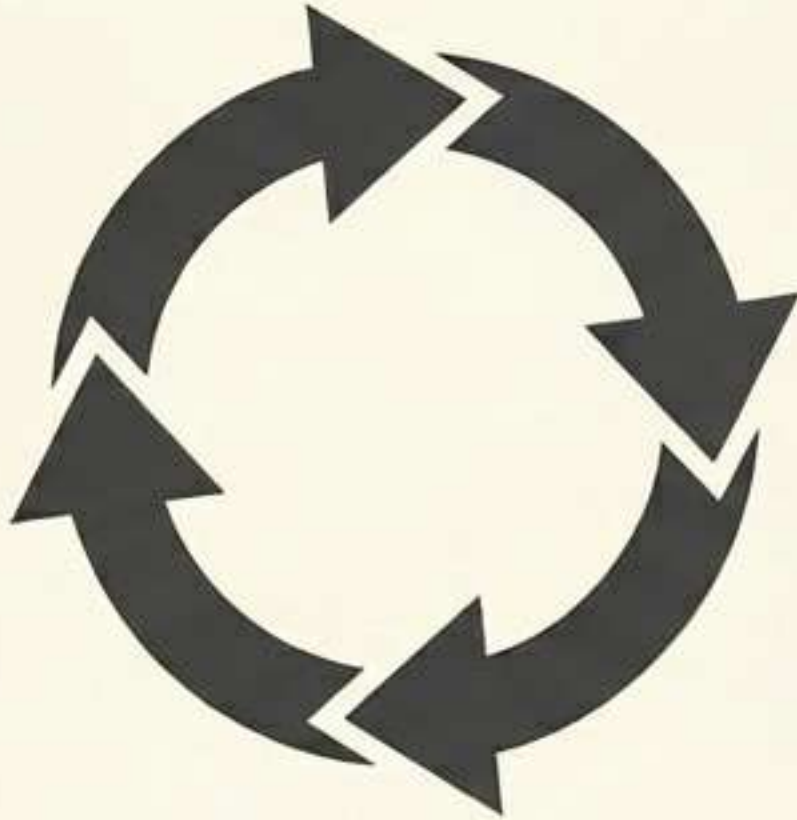
The Primary Benefit: Computational Efficiency



- Decomposing matrix A is computationally expensive.
- However, once factorised, L and U can be reused to solve multiple systems of $Ax = b$.
- Solving the triangular systems ($Ly=b$ and $Ux=y$) is highly efficient.

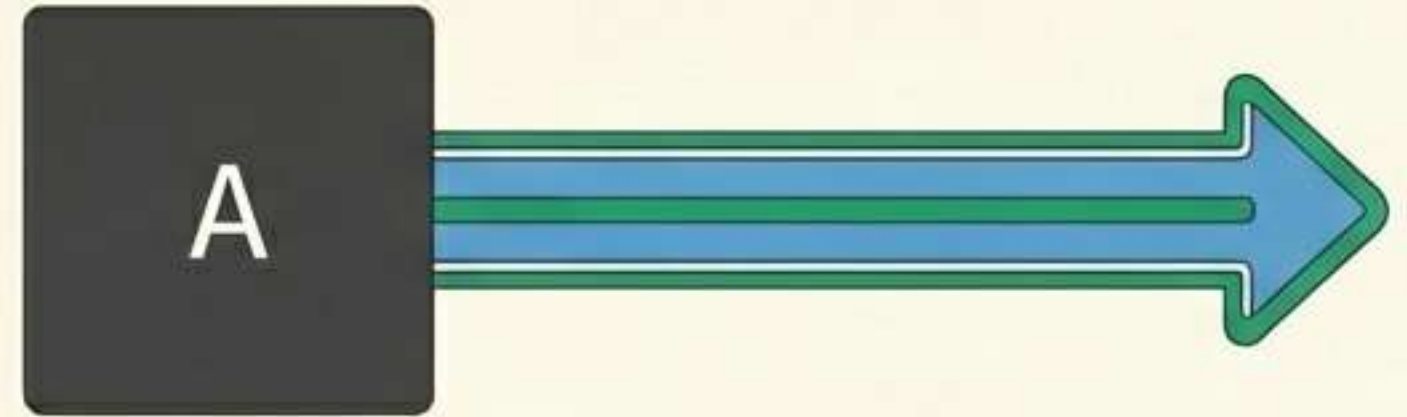
LU Factorisation vs. Gaussian Elimination

Gaussian Elimination



Must perform the entire row reduction process from scratch for every new vector $\mathbf{b}$.

LU Decomposition



Factorises A just once. Reuses L and U for any new vector $\mathbf{b}$.

The LU Decomposition Cheat Sheet

The Factorisation

$$A = LU$$



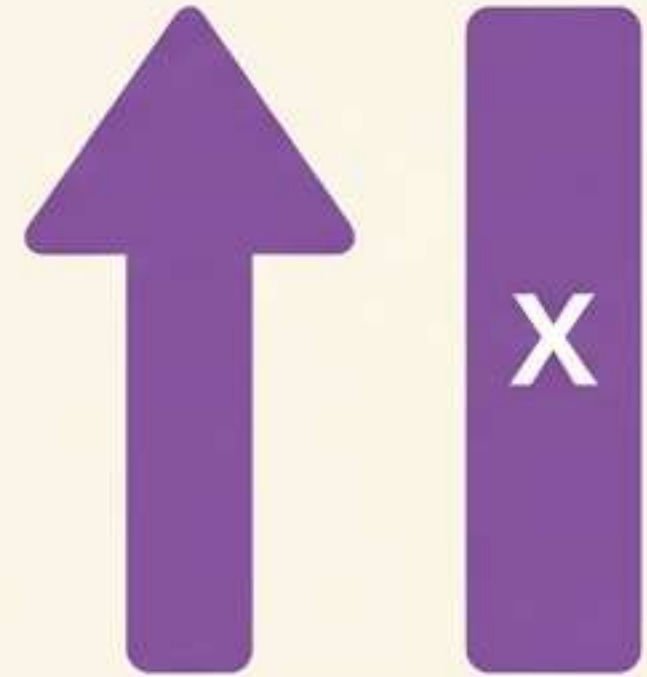
Step 1 (Forward)

$$Ly = b$$



Step 2 (Backward)

$$Ux = y$$



LU Decomposition transforms the computationally heavy task of solving $Ax = b$ into two fast, sequential steps, unlocking massive efficiency for large-scale data science applications.